

OE-EC704A

Web Technology

Combined Module Notes

Modules 1 – 5

Semester VII • 3L:0T:0P • 3 Credits

CGEC | B.Tech ECE (2023–27)

Biraj Sarkar

Disclaimer / Study Guide Notice:

These are condensed **Short Notes** focusing on core theory, key formulas, block/circuit diagrams, and standard viva Q&As for university exam revision. Exhaustive numerical practice problems and derivation edge cases are not covered. This serves as a quick-revision reference guide for students.

Contents

Module 1: Introduction to Web Development Basics	4
1 Introduction to Web Development Basics	4
1.1 HTML vs. XML	4
2 HTML Basics and Document Structure	5
2.1 Standard Skeleton of an HTML5 Document	5
2.2 Block-level vs. Inline Elements	6
2.3 Essential HTML Components	6
3 CSS (Cascading Style Sheets)	7
3.1 Types of CSS Integration	7
3.2 CSS Selectors	7
3.3 The CSS Box Model	8
Module 2: Java Execution Environment	12
1 Java Execution Environment	12
1.1 JDK vs. JRE vs. JVM	12
2 Java Syntax and Control Flow	13
2.1 Primitive vs. Reference Data Types	13
2.2 Memory Allocation Model	13
2.3 Control Flow Statements	14
3 Class Fundamentals and Variables	14
3.1 Static vs. Instance Members	14
3.2 Inner Classes	15
Module 3: Core Object-Oriented Programming (OOP) Principles	19
1 Core Object-Oriented Programming (OOP) Principles	19
2 Inheritance and Reuse Models	20
2.1 Composition vs. Inheritance	20
3 Polymorphism and Runtime Resolution	21
3.1 Method Overloading vs. Overriding	21
3.2 Dynamic Method Dispatch	21
4 Interfaces, Abstract Classes, and Access Modifiers	22
4.1 Abstract Class vs. Interface	22
4.2 Access Modifiers	22
5 Object Class and SOLID Design Principles	22
5.1 The Root Object Class	22
5.2 SOLID Design Principles	22

Module 4: Java Exception Handling Mechanism	27
1 Java Exception Handling Mechanism	27
1.1 Checked vs. Unchecked Exceptions	28
2 Java String Handling	28
2.1 String vs. StringBuffer	28
2.2 String and StringBuffer Constructors	28
2.3 Core String Operations	29
2.4 Utility Conversions: <code>toString()</code> and <code>valueOf()</code>	30
3 Java Input/Output (I/O) Streams	31
3.1 Byte Stream vs. Character Stream	31
4 Java Collections Framework	31
4.1 ArrayList vs. LinkedList	31
Module 5: Java Applets	36
1 Java Applets	36
1.1 Applet Life Cycle	36
1.2 Applet Display and Status Methods	37
1.3 HTML Applet Tag and Parameters	37
1.4 Applet vs. Standard Application	38
2 Delegation Event Model	38
2.1 Adapter Classes	39
3 AWT (Abstract Window Toolkit)	39
3.1 AWT vs. Swing	39
3.2 Layout Managers	39

Module 1

Introduction to Web Development Basics

1. Introduction to Web Development Basics

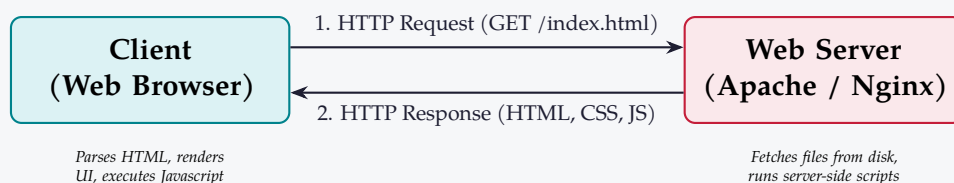
Modern web development relies on a client-server architecture. Web pages are hosted on web servers and requested by clients (web browsers) via HTTP/HTTPS protocols.

Client-Server Web Architecture

The **Client-Server Model** is a distributed application structure that partitions tasks between resource providers (**servers**) and service requesters (**clients**). In web technology:

- **Client (Web Browser):** Sends HTTP requests, parses HTML/CSS/JS, and renders the user interface.
- **Server (Web Server):** Listens for incoming requests on ports 80 (HTTP) or 443 (HTTPS), processes logic, and returns the requested resources.

Web Architecture: Client-Server Flow



1.1 HTML vs. XML

HTML and XML are both markup languages based on SGML, but they serve fundamentally different purposes:

Feature	HTML (HyperText Markup Language)	XML (eXtensible Markup Language)
Primary Purpose	Displaying and formatting data to users.	Transporting, storing, and structuring data.
Tags	Predefined tags (e.g., <html>, <p>).	Custom/user-defined tags based on data.
Syntax Strictness	Forgiving (errors don't crash rendering).	Extremely strict (well-formed check required).
Case Sensitivity	Case-insensitive (<DIV> and <div>).	Strictly case-sensitive.
Closing Tags	Optional for some tags (e.g., ,).	Mandatory for all tags.

2. HTML Basics and Document Structure

HTML — HyperText Markup Language

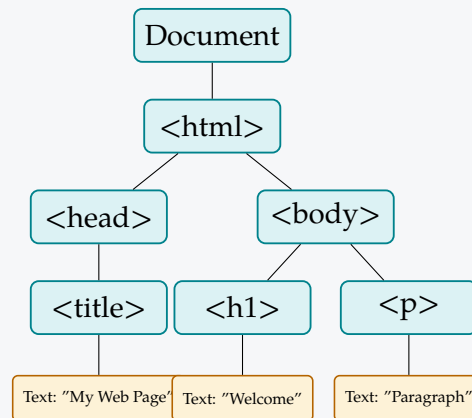
HTML is the standard markup language used to create the structure of web pages. It uses a set of hierarchical elements represented by tags to organize headings, paragraphs, tables, images, and other media elements.

2.1 Standard Skeleton of an HTML5 Document

Every HTML document must follow a standard structural hierarchy:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>My Web Page</title>
  </head>
  <body>
    <h1>Welcome to Web Technology</h1>
    <p>This is a paragraph.</p>
  </body>
</html>
```

HTML Document Object Model (DOM) Tree



2.2 Block-level vs. Inline Elements

HTML elements are categorized based on their default layout behaviors:

Feature	Block-level Elements	Inline Elements
Layout Behavior	Starts on a new line and takes up full width.	Stays on the same line and occupies only needed width.
Margins & Padding	Fully respected on all four sides.	Top and bottom margins are ignored.
Containment	Can contain block or inline elements.	Can only contain inline elements / text.
Examples	<code><div></code> , <code><p></code> , <code><h1></code> – <code><h6></code> , <code></code> , <code></code> , <code><table></code>	<code></code> , <code><a></code> , <code></code> , <code></code> , <code></code> , <code><input></code>

2.3 Essential HTML Components

1. Lists: HTML supports three list types:

- **Unordered List (`<ul>`):** Bulleted list items using the `<li>` tag.
- **Ordered List (`<ol>`):** Numbered list items with customizable types (numeric, alphabetic, roman).
- **Description List (`<dl>`):** Pairs of terms (`<dt>`) and descriptions (`<dd>`).

2. Links (Anchors `<a>`): Used for navigating between pages or sections:

- **Absolute Link:** Links to a fully qualified URL (e.g., `href="https://google.com"`).
- **Relative Link:** Links to a local resource relative to the file path (e.g., `href="about.html"`).
- **Email Link:** Triggers the client's email client using the mailto protocol (e.g., `href="mailto:test@test.com"`).
- **Anchor Targets:** Uses `target="_blank"` to open in a new tab or `target="_self"` for the same frame.

3. Tables (`<table>`): Formats data in rows and columns:

- **Rowspan:** Extends a cell vertically across multiple rows (e.g., `<td rowspan="2">`).
- **Colspan:** Extends a cell horizontally across multiple columns (e.g., `<td colspan="3">`).

4. **Forms (<form>):** Captures and submits user inputs to a server. Key attributes:

- **action:** The URL of the server-side endpoint that handles the submitted form data.
- **method:** The HTTP method used to send data (GET or POST).
- **Input Types:** Text, password, radio buttons, checkboxes, email, submit, file, number.

5. **Frames (<iframe>):** Inline frames embed external HTML documents inside the current page.

3. CSS (Cascading Style Sheets)

CSS — Cascading Style Sheets

CSS is a style sheet language used to describe the presentation, design, formatting, and layout of a document written in HTML. It separates structure from style, allowing consistent look-and-feel across multiple web pages.

3.1 Types of CSS Integration

CSS Type	Syntax Example	Advantages	Disadvantages
Inline CSS	<code><p style="color:red;"></code>	High priority; quick overrides.	Messy; violates separation of concerns.
Internal CSS	<code><style> p { color: red; } </style></code>	Contained inside single page; no extra HTTP request.	Reusability limit across multiple pages.
External CSS	<code><link rel="stylesheet" href="style.css"></code>	High reuse; caching support; clean separation.	Requires extra network HTTP request.

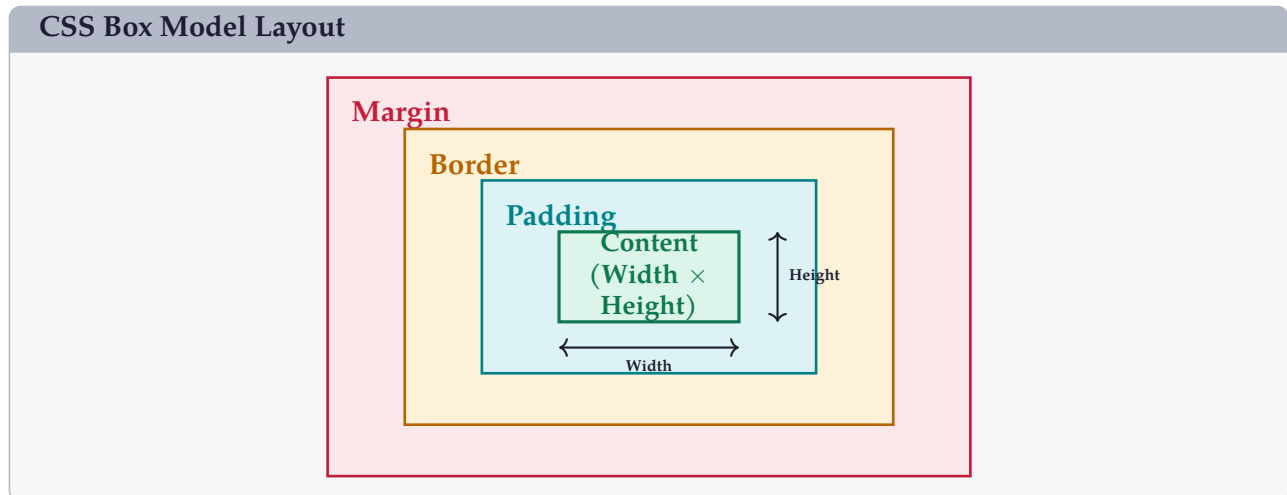
3.2 CSS Selectors

CSS selectors target specific HTML elements for styling:

- **Element Selector:** Targets all elements of a type (e.g., `p { color: red; }`).
- **Class Selector:** Targets elements with a matching class name, preceded by a dot (e.g., `.container { width: 80%; }`).
- **ID Selector:** Targets a single element with a matching ID, preceded by a hash (e.g., `#header { height: 60px; }`). ID must be unique within the page.
- **Universal Selector:** Targets every element in the DOM using asterisk (e.g., `* { margin: 0; }`).

3.3 The CSS Box Model

Every element in HTML is rendered as a rectangular box. The CSS Box Model defines how the size and spacing of this box are calculated.



The dimensions of an element are computed based on the `box-sizing` property:

- **content-box (Default):** Total Width = width + left/right padding + left/right border + left/right margin.
- **border-box:** Total Width = width + left/right margin (padding and border are absorbed inside the specified width).

Quick Revision — Module 1

Web Development Basics (HTML & CSS) | OE-EC704A

Core HTML Tags & CSS Syntax

HTML Form / Tag	Syntax and Key Attributes
Hyperlink Anchor	<code>Link Text</code>
Embedded Image	<code></code>
HTML Table Structure	<code><table> <tr> <td colspan="2">Data</td> </tr> </table></code>
HTML Form Tag	<code><form action="/login" method="POST"> inputs... </form></code>
CSS Selector Type	Syntax and Example Usage
CSS Syntax Rule	<code>selector { property: value; }</code>
Class Styling	<code>.button { background-color: blue; padding: 10px; }</code>
ID Styling	<code>#main-nav { display: flex; justify-content: space-between; }</code>
Universal Reset	<code>* { box-sizing: border-box; margin: 0; padding: 0; }</code>

One-Line Definitions

- **HTML:** Structured markup language defining content hierarchy of a web document.
- **CSS:** Style sheet language used for presenting and laying out HTML elements.
- **DOM (Document Object Model):** An API representing an HTML document as a logical tree structure.
- **Tag:** A code marker representing the start or end of an HTML element (e.g., `<p>`).
- **Element:** A complete component consisting of start tag, content, and end tag.
- **Attribute:** Name-value pairs inside HTML tags that modify their behavior or supply metadata.
- **Viewport:** The visible area of a web page on a client device screen.
- **URL (Uniform Resource Locator):** The global reference address of a web resource on the Internet.

Frequently Asked / Viva Questions

Q1. What is the difference between GET and POST methods in forms?

GET appends form data directly to the URL as query parameters, rendering it visible and bookmarkable. It is intended for idempotent requests. POST sends data inside the HTTP message body, making it secure for sensitive parameters like passwords.

Q2. Why is DOCTYPE html declared at the beginning of an HTML file?

Frequently Asked / Viva Questions (contd.)

It is a standard preamble instructing the client browser to render the document in standards-compliant mode rather than quirks mode.

Q3. What is the difference between `span` and `div` tags?

`div` is a block-level container element that starts on a new line. `span` is an inline element used to style specific words or content inline without inserting page breaks.

Q4. How does CSS specificity calculate which style to apply?

Specificity is calculated as a numeric weight: inline styles hold the highest priority (1000), followed by ID selectors (100), class/attribute selectors (10), and element selectors (1).

Q5. What is the difference between relative and absolute positioning in CSS?

Relative positioning offsets an element relative to its normal position in the document flow. Absolute positioning moves the element relative to its closest parent with non-static positioning.

Q6. What does the `box-sizing: border-box` property do?

It alters the CSS Box Model calculation so that borders and paddings are included in the element's declared width and height, preventing layout breakage.

Q7. What are `colspan` and `rowspan` attributes in HTML tables?

`colspan` merges cells horizontally across columns. `rowspan` merges cells vertically across rows.

Q8. What is semantic HTML and why is it important?

Semantic HTML uses tags that explain their purpose clearly (e.g., `<header>`, `<nav>`, `<article>`, `<footer>`) instead of generic `<div>` blocks. It enhances accessibility and SEO search indexing.

Q9. How do you link CSS files to an HTML document?

Using the HTML `<link>` element inside the `<head>` block: `<link rel="stylesheet" href="style.css">`.

Q10. What are CSS pseudoclasses and how are they used?

Pseudoclasses define special states of elements (e.g., `:hover` for mouse hover, `:active` for clicks, `:focus` for keyboard selection).

Form Submission: GET vs. POST comparison

Metric	GET Method	POST Method
Data Location	Appended to URL (Query parameters).	Inside HTTP Request Body.
Security	Poor (exposed in browser history/logs).	Higher (hidden from URL path).
Payload Limits	Limited (typically 2048 characters).	No architectural payload size limit.
Caching	Allowed/Cached by proxy servers.	Never cached.
Idempotency	Idempotent (submitting multiple times is safe).	Non-idempotent (resubmitting duplicates actions).

Module 2

Java Execution Environment

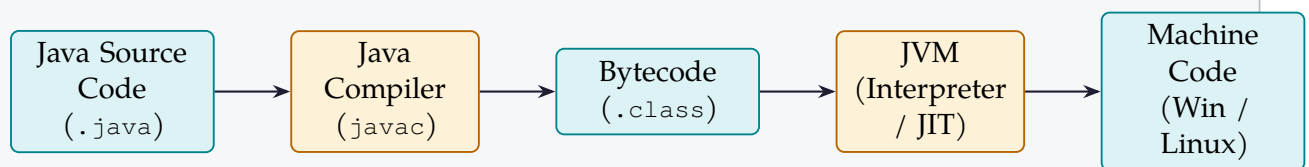
1. Java Execution Environment

Java uses a hybrid execution model combining compilation and interpretation. This ensures its core principle: **Write Once, Run Anywhere (WORA)**.

Bytecode and Platform Independence

- **Bytecode:** A highly optimized set of machine instructions designed for execution by the JVM. It is stored in a `.class` file after compilation.
- **Platform Independence:** Java source code compiles into platform-independent bytecode. However, the **JVM itself is platform-dependent** because it must translate bytecode into the host system's native machine instructions.

Java Compile-and-Run Execution Flow



1.1 JDK vs. JRE vs. JVM

Understanding the boundary between JVM, JRE, and JDK is crucial:

JDK, JRE, and JVM Containment Relationship

JDK (Java Development Kit)Development Tools: `javac`, `jar`, `jdb`, `javadoc`**JRE (Java Runtime Environment)**Libraries: `rt.jar`, Core API runtime support**JVM (Java Virtual Machine)**

Class Loader, Execution Engine (Interpreter + JIT), GC, Memory Manager

Metric	JVM (Virtual Machine)	JRE (Runtime)	JDK (Development)
Composition	Classloader, Execution Engine, Garbage Collector.	JVM + Core class libraries (<code>rt.jar</code>) + plugins.	JRE + Compiler (<code>javac</code>) + tools (<code>jar</code> , <code>jdb</code>).
Purpose	Execution of compiled bytecode.	Running Java applications.	Developing and running Java apps.
Target User	None (internal engine).	End users/clients.	Developers.

2. Java Syntax and Control Flow

2.1 Primitive vs. Reference Data Types

Java enforces static type checking and groups types into two primary categories:

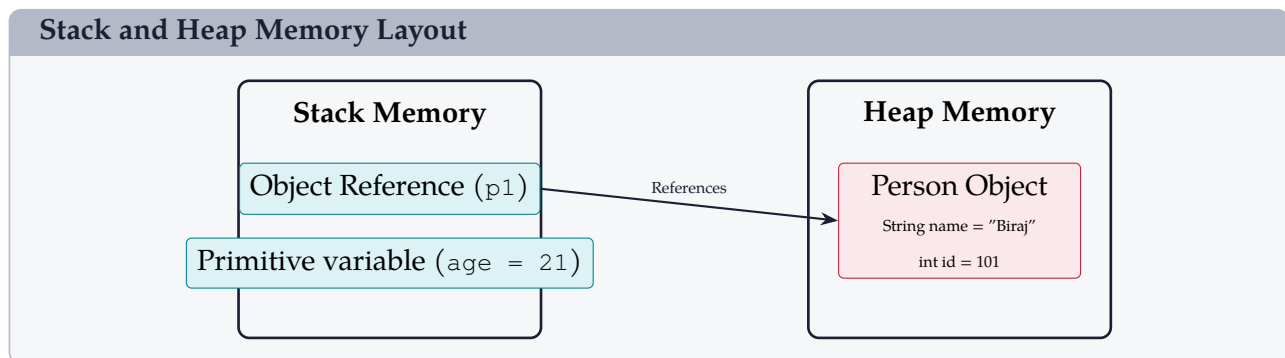
Feature	Primitive Data Types	Reference Data Types
Allocation	Stored directly on the stack memory.	Object on Heap; address reference on Stack.
Default Value	0, 0.0, or false (never null).	null by default.
Size	Fixed sizing (e.g., <code>int</code> : 32-bit, <code>double</code> : 64-bit).	Depends on the host system architecture.
Examples	<code>byte</code> , <code>short</code> , <code>int</code> , <code>long</code> , <code>float</code> , <code>double</code> , <code>char</code> , <code>boolean</code>	Classes, interfaces, arrays, strings.

2.2 Memory Allocation Model

Java allocates memory across two distinct spaces during runtime:

- **Stack Memory:** Stores primitive variables and local references. Uses LIFO allocation.

- **Heap Memory:** Dynamic memory pool that hosts all runtime objects and instance variables.



2.3 Control Flow Statements

Java supports selection (`if-else`, `switch`), iteration (`for`, `while`, `do-while`), and jump (`break`, `continue`, `return`) statements:

- **break:** Exits the innermost loop or switch statement immediately.
- **continue:** Skips the current iteration and jumps directly to the next condition evaluation.
- **Labeled Statements:** Loops can be labeled to specify which outer loop a `break` or `continue` targets (e.g., `outerLoop: for(...)`).

3. Class Fundamentals and Variables

Class, Object, and Constructor

- **Class:** A blueprint or template that defines the structure and behaviors (fields and methods) of objects.
- **Object:** A physical instance of a class, holding state values and responding to message calls.
- **Constructor:** A special block of code resembling a method, invoked during object instantiation via the `new` keyword. It must match the class name exactly and has no return type.

3.1 Static vs. Instance Members

Members marked with the `static` keyword behave differently from normal instance variables:

Feature	Static Members	Instance Members
Memory Allocation	Allocated once in the Metaspace/Method area.	Allocated inside each object on the Heap.
Access Syntax	Accessed via class name (e.g., <code>Math.sqrt()</code>).	Requires an object instance to access.
Lifecycle	Lives from class load to class unload.	Tied to object lifetime; garbage collected.
Context	Cannot reference <code>this</code> or <code>super</code> .	Full access to static and instance members.

3.2 Inner Classes

An inner class is a class declared inside another outer class:

- **Member Inner Class:** Non-static nested class. Holds a reference to the outer class instance and can access all its private members directly.
- **Static Nested Class:** Declared static. Cannot access non-static outer members directly.
- **Anonymous Inner Class:** A class without a name defined inline to implement interfaces or extend abstract classes.

Quick Revision — Module 2

Introduction to Java Programming & OOP | OE-EC704A

Java Class & Loop Syntax

Concept	Syntax and Structural Format
Class Declaration	<code>class Person { String name; Person(String n) { name = n; } }</code>
Main Method Signature	<code>public static void main(String[] args) { ... }</code>
Object Instantiation	<code>Person p1 = new Person("Biraj");</code>
Static Method Call	<code>ClassName.staticMethodName();</code>
Loop Statements	Conditional and Iterative Formats
for Loop	<code>for(initialization; condition; increment/decrement) { ... }</code>
while Loop	<code>while(condition) { statements; }</code>
do-while Loop	<code>do { statements; } while(condition);</code>
Labeled Break	<code>outer: for(...) { if(cond) break outer; }</code>

One-Line Definitions

- **JVM (Java Virtual Machine):** An abstract machine that parses and executes compiled Java bytecode.
- **Bytecode:** Standard intermediate instruction format generated by the compiler.
- **JDK:** Comprehensive toolset containing JRE and compiler scripts for developing Java apps.
- **JRE:** Run runtime libraries and JVM package required to execute Java bytecode.
- **Class:** User-defined blueprint declaring structural fields and behavior routines.
- **Object:** An active, stateful runtime instance of a class allocated on the heap.
- **Constructor:** Instantiation routine that initializes object state variables during creation.
- **Garbage Collector:** Automatic JVM system thread that frees heap memory by sweeping unreferenced objects.

Frequently Asked / Viva Questions

Q1. Why is Java platform-independent but JVM is platform-dependent?

Java code compiles to bytecode which runs on any JVM, regardless of the host. However, the JVM must be tailored to the specific OS architecture to translate that bytecode to native machine instructions.

Frequently Asked / Viva Questions (contd.)**Q2. What is the difference between `path` and `classpath` variables?**

`path` is an OS environment variable locating command-line executables like `javac` and `java`. `classpath` instructs the JVM where to look for compiled user classes and libraries.

Q3. Can a class have multiple constructors? If so, how are they distinguished?

Yes, this is known as constructor overloading. They are distinguished by their signatures: the number, type, and order of parameters.

Q4. What is the default constructor and when is it generated?

If no constructor is declared, the compiler automatically generates a no-argument default constructor that initializes fields to default values. If any constructor is declared, this default constructor is not created.

Q5. What is the difference between `break` and `continue` statements?

`break` exits a loop block immediately, while `continue` skips only the current iteration and evaluates the loop condition for the next cycle.

Q6. What is the purpose of the `static` keyword in Java?

It binds variables or methods to the class itself, rather than to instances. They are loaded once in class metadata space.

Q7. Can a static method access non-static instance variables? Why?

No. Static methods run without any object instance context, so they do not have access to the `this` reference necessary to locate instance variables.

Q8. What is the difference between pass-by-value and pass-by-reference in Java?

Java is strictly pass-by-value. For primitives, a copy of the value is passed. For reference types, a copy of the heap reference address is passed, not the object itself.

Q9. What is the purpose of garbage collection in Java?

It runs asynchronously to track and reclaim heap memory occupied by objects that have become unreachable from active threads.

Q10. Why does Java not support global variables?

To prevent global namespace pollution and promote modular encapsulation, Java requires every variable to reside inside a class scope.

Java Iteration Statement Comparisons

Feature	for Loop	while Loop	do-while Loop
Loop Structure	Counter-based iteration.	Condition-controlled iteration.	Exit-controlled iteration.
Condition Check	Evaluated at start.	Evaluated at start.	Evaluated at end.
Min. Runs	0 runs if condition fails.	0 runs if condition fails.	Guaranteed at least 1 run.
Best Use Case	Iterating when count is known.	Loop until event triggers.	Interactive menus.

Module 3

Core Object-Oriented Programming (OOP) Principles

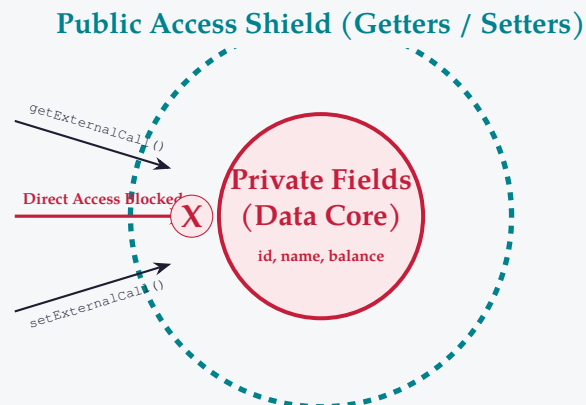
1. Core Object-Oriented Programming (OOP) Principles

Java is a pure object-oriented programming language designed around four fundamental concepts.

The Four Pillars of Object-Oriented Programming

1. **Encapsulation (Data Hiding):** Wrapping variables and methods together into a single unit (class) while restricting direct outside access using the `private` modifier and exposing access via `public` getters and setters.
2. **Abstraction:** Hiding internal execution complexity and showing only the essential feature interfaces to the outer user. Implemented using **Abstract Classes** and **Interfaces**.
3. **Inheritance:** The mechanism by which one class (subclass/child) inherits the state variables and functional behaviors of another class (superclass/parent) to facilitate code reusability.
4. **Polymorphism:** The ability of a reference to take on multiple forms. It is divided into:
 - **Compile-time (Static):** Resolved by compiler signatures via **Method Overloading**.
 - **Runtime (Dynamic):** Resolved at execution via **Method Overriding** and Dynamic Method Dispatch.

Encapsulation Boundary & Data Protection Shield



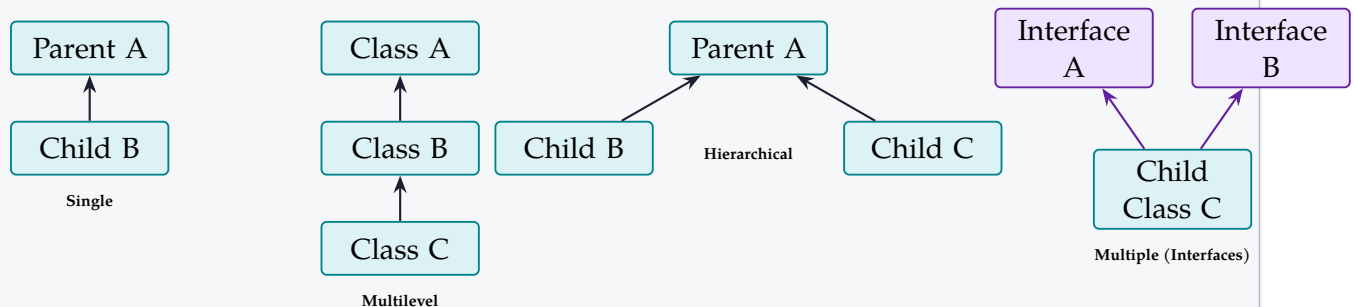
2. Inheritance and Reuse Models

Inheritance enables hierarchical classifications. In Java, classes are extended using the `extends` keyword.

Inheritance Characteristics

- **Single Inheritance Only:** Java classes can only extend **one** parent class to prevent the diamond problem (ambiguity under duplicate variable/method signatures).
- **The `super` Keyword:** Used to invoke the parent class constructor (must be the first statement in the subclass constructor), access overridden parent methods, or read hidden parent variables.

Inheritance Trees supported in Java



2.1 Composition vs. Inheritance

Design patterns advocate favoring object composition over class inheritance:

Metric	Composition (Has-A Relationship)	Inheritance (Is-A Relationship)
Coupling Level	Loose coupling; internal details remain private.	Tight coupling; breaks encapsulation boundary.
Flexibility	Behaviors can change dynamically at runtime.	Static; behavior fixed at compilation time.
Design Reuse	Reuses functional components as class members.	Reuses code by expanding a class hierarchy.
Example	Car has an Engine instance.	Car is a subclass of Vehicle.

3. Polymorphism and Runtime Resolution

3.1 Method Overloading vs. Overriding

These represent static and dynamic polymorphism:

Feature	Method Overloading (Static)	Method Overriding (Dynamic)
Class Context	Occurs within the same class definition.	Occurs across child-parent subclass hierarchy.
Method Signature	Name must be same; parameters must differ.	Signature (parameters, returns) must match.
Resolution Time	Compile-time (static binding).	Runtime (dynamic binding).
Keywords	None required.	Assisted by @Override annotation.

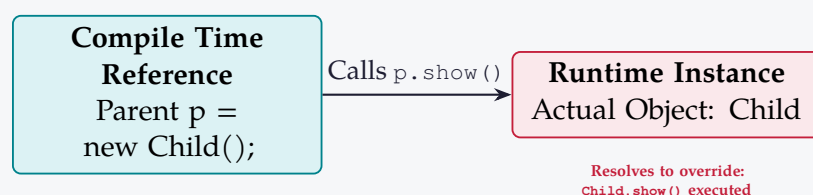
3.2 Dynamic Method Dispatch

Dynamic Method Dispatch is the mechanism by which a call to an overridden method is resolved at runtime rather than compile-time.

Dynamic Method Dispatch Rule

When an overridden method is called through a superclass reference, Java determines which version of the method to execute based on the **type of the actual object** being referred to at runtime, not the type of the reference variable.

Dynamic Method Dispatch Flow



4. Interfaces, Abstract Classes, and Access Modifiers

4.1 Abstract Class vs. Interface

These constructs provide different levels of abstraction:

Feature	Abstract Class	Interface
Multiple Inheritance	Allowed to inherit only one abstract parent.	A class can implement multiple interfaces.
State Variables	Can contain instance variables (stateful).	Variables must be <code>public static final</code> .
Methods	Concrete methods and abstract methods.	Abstract methods (pre-Java 8), defaults allowed.
Keywords	Declared with <code>abstract class</code> and <code>extends</code> .	Declared with <code>interface</code> and <code>implements</code> .

4.2 Access Modifiers

Java regulates visibility using four access levels:

- **private:** Visible only within the declaring class.
- **Default (no modifier):** Visible only to classes inside the same package.
- **protected:** Visible to the same package and subclass child files in external packages.
- **public:** Accessible from any class anywhere in the project.

5. Object Class and SOLID Design Principles

5.1 The Root Object Class

Every Java class implicitly extends `java.lang.Object`. It defines essential methods:

- **toString():** Returns a string representation of the object (`ClassName@HexHashCode`).
- **equals(Object obj):** Compares reference equality. Overridden to compare values.
- **hashCode():** Returns an integer hash code mapped to the object's memory address.
- **clone():** Creates a field-by-field copy of the current object.

5.2 SOLID Design Principles

These principles ensure Java classes are maintainable:

- **Single Responsibility Principle (SRP):** A class must have exactly one reason to change.
- **Open/Closed Principle (OCP):** Software entities should be open for extension, closed for modification.
- **Liskov Substitution Principle (LSP):** Derived classes must be fully substitutable for their base classes.

- **Interface Segregation Principle (ISP):** Clients should not be forced to depend on interfaces they do not use.
- **Dependency Inversion Principle (DIP):** Depend on abstractions, not concrete implementations.

Quick Revision — Module 3

OOP Principles, Inheritance, Polymorphism, Packages & Interfaces | OE-EC704A

Inheritance, Packages & Interface Syntax

Concept	Syntax and Structural Format
Subclass Extension	<code>class Child extends Parent { ... }</code>
Super Constructor Call	<code>Child(int val) { super(val); ... }</code>
Interface Declaration	<code>interface Flyable { void fly(); }</code>
Interface Implementation	<code>class Bird implements Flyable { public void fly() { ... } }</code>
Package Syntax	Access and Import Declarations
Package Header	<code>package com.cgec.webtech;</code>
Import Header	<code>import com.cgec.webtech.MyClass;</code>
Abstract Method	<code>abstract void calculateScore();</code>
Constant in Interface	<code>int MAX_RETRY = 5; // Public static final by default</code>

One-Line Definitions

- **Inheritance:** The mechanism by which a subclass inherits variables and methods from a superclass.
- **Method Overloading:** Declaring methods with the same name but different parameters in one class.
- **Method Overriding:** Declaring a method in a subclass with the same signature as in the parent class.
- **Dynamic Method Dispatch:** Dynamic binding resolution that determines overridden method calls at runtime.
- **Abstract Class:** A class that cannot be instantiated and may contain abstract methods.
- **Interface:** A contract defining empty method signatures that implementing classes must satisfy.
- **Package:** A namespace group containing related classes, interfaces, and packages.
- **SOLID:** A set of five object-oriented design principles for readable and maintainable software.

Frequently Asked / Viva Questions

Q1. Why does Java not support multiple inheritance with classes?

To avoid the diamond problem. If class C extends both class A and class B, and both parents define a method with the same signature `show()`, class C cannot determine

Frequently Asked / Viva Questions (contd.)

which parent version to inherit.

Q2. How does Java support multiple inheritance then?

Java allows a class to implement multiple interfaces. Since interfaces do not hold instance state and their methods were traditionally empty, there is no duplicate implementation ambiguity.

Q3. What is the difference between `this()` and `super()` constructor calls?

`this()` invokes another constructor within the same class. `super()` invokes a constructor of the parent superclass. Both must be the first line of their respective constructors.

Q4. Can an abstract class have constructors? Why?

Yes. Although an abstract class cannot be instantiated directly, its constructor is called during subclass instantiation via `super()` to initialize its fields.

Q5. What is the difference between `final` class, `final` method, and `final` variable?

A `final` class cannot be inherited. A `final` method cannot be overridden. A `final` variable behaves as a constant and cannot be reassigned.

Q6. What is the contract between `equals()` and `hashCode()`?

If two objects are equal according to `equals()`, they must return the same integer hash code from `hashCode()`. If overridden, both must be updated to preserve this rule.

Q7. What is the difference between composition and inheritance?

Inheritance represents an IS-A relationship and creates tight coupling. Composition represents a HAS-A relationship, wrapping objects to provide loose coupling.

Q8. How does the access level of a subclass method override compare to the parent class method?

An overriding method cannot restrict the access level of the overridden parent method. For example, a `protected` method in a parent class can only be overridden as `protected` or `public`.

Q9. What are default methods in interfaces and why were they introduced in Java 8?

Default methods have an implementation inside the interface using the `default` keyword. They allow adding new methods to interfaces without breaking existing implementing classes.

Q10. What is the difference between compile-time and runtime polymorphism?

Compile-time polymorphism (overloading) resolves method calls during compilation using static binding. Runtime polymorphism (overriding) resolves method calls during execution using dynamic binding.

Java Access Modifier Visibility Matrix

Modifier	Same Class	Same Package	Subclass	Everywhere
private	Yes	No	No	No
Default	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

Module 4

Java Exception Handling Mechanism

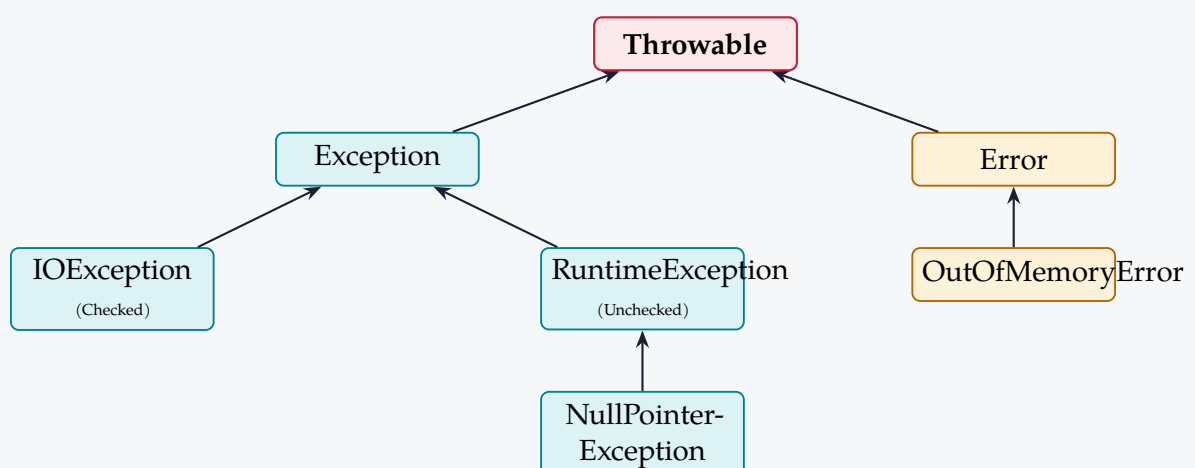
1. Java Exception Handling Mechanism

An exception is an event that disrupts the normal flow of instruction execution. Java uses a structured exception handling system based on five keywords: `try`, `catch`, `throw`, `throws`, and `finally`.

Exception Types

- **Checked Exceptions (Compile-time):** Exceptions that must be declared in the method signature or handled in a try-catch block. Verified by the compiler (e.g., `IOException`, `SQLException`).
- **Unchecked Exceptions (Runtime):** Exceptions that extend `RuntimeException`. Not verified by the compiler (e.g., `NullPointerException`, `ArithmeticException`).
- **Errors:** Serious conditions that applications should not try to catch. They represent system failures (e.g., `OutOfMemoryError`, `StackOverflowError`).

Java Exception Class Hierarchy



1.1 Checked vs. Unchecked Exceptions

Feature	Checked Exceptions	Unchecked Exceptions
Compiler Checks	Enforced at compilation.	Not verified by the compiler.
Base Superclass	Inherits from <code>java.lang.Exception</code> .	Inherits from <code>java.lang.RuntimeException</code> .
Resolution Rule	Must use <code>try-catch</code> or declare <code>throws</code> .	Recommended to write defensive checks.
Examples	<code>FileNotFoundException</code> , <code>IOException</code>	<code>ArithmeticException</code> , <code>NullPointerException</code>

2. Java String Handling

Java represents sequences of characters using the `String` class.

String Immutability

Java `String` objects are **immutable**, meaning their state cannot be modified after creation.

- **String Pool:** When a string literal is declared, the JVM checks if it exists in the String Constant Pool (SCP) on the heap. If present, it reuses the reference, saving heap memory.
- **Why Immutable?** Ensures thread safety, security (parameters like DB connection URLs cannot be altered), and enables string pool caching.

2.1 String vs. StringBuffer

Metric	String Class	StringBuffer Class
Mutability	Immutable (read-only once created).	Mutable (can append or modify content).
Performance	Slower for frequent concatenations (creates new objects).	Fast and memory efficient (modifies local buffer).
Thread Safety	Naturally thread-safe due to immutability.	Thread-safe; methods are synchronized.
Heap Allocation	SCP or standard heap space.	Standard heap space only.

2.2 String and StringBuffer Constructors

Both classes provide multiple constructors to instantiate character sequences:

Constructor	Signature	Description
Default String	<code>String()</code>	Initializes a newly created, empty String object.
Char Array String	<code>String(char[] value)</code>	Allocates a new String representing the char array's contents.
String Copier	<code>String(String original)</code>	Initializes a copy of the argument string.
Buffer Converter	<code>String(StringBuffer sb)</code>	Converts a mutable StringBuffer into an immutable String.
Default Buffer	<code>StringBuffer()</code>	Creates an empty buffer with an initial capacity of 16 characters.
Capacity Buffer	<code>StringBuffer(int cap)</code>	Creates an empty buffer with the specified initial capacity.
String Buffer	<code>StringBuffer(String str)</code>	Creates a buffer initialized with the specified string content.

2.3 Core String Operations

The String class provides a rich set of methods for manipulating character sequences. Because String is immutable, all modifying methods return a **new** String instance representing the result.

Operation	Key Method Signature	Functional Description
Extraction	<code>char charAt(int index)</code>	Returns the char value at the specified index (0 to length – 1).
	<code>int length()</code>	Returns the number of characters in the string.
Comparison	<code>boolean equals(Object obj)</code>	Compares string values for character-by-character equality.
	<code>boolean equalsIgnoreCase(String s)</code>	Value comparison ignoring case differences.
	<code>int compareTo(String anotherString)</code>	Lexicographically compares two strings (returns 0 if equal, negative if less, positive if greater).
Searching	<code>int indexOf(String str)</code>	Returns the index of the first occurrence of the substring (or –1).
	<code>int lastIndexOf(String str)</code>	Returns the index of the last occurrence of the substring.
	<code>boolean contains(CharSequence s)</code>	Returns true if the string contains the specified sequence.
Modifying	<code>String substring(int begin, int end)</code>	Extracts a new substring from <code>begin</code> (inclusive) to <code>end</code> (exclusive).
	<code>String concat(String str)</code>	Appends the specified string to the end (equivalent to the + operator).
	<code>String replace(char old, char new)</code>	Returns a new string with all occurrences of <code>old</code> replaced by <code>new</code> .
	<code>String trim()</code>	Returns a copy with leading and trailing whitespaces omitted.

2.4 Utility Conversions: `toString()` and `valueOf()`

Java provides built-in mechanisms to convert objects and primitive values into string representations:

- The `toString()` Method:** Defined in the root `Object` class. Every class inherits it, and standard library classes (like `String`, `StringBuffer`, `ArrayList`) override it to return a readable value representation.
 - When an object reference is printed via `System.out.println(obj)`, the JVM implicitly invokes `obj.toString()`.
 - By default, it returns: `getClass().getName() + "@" + Integer.toHexString(hashCode())`.
- The Static `String.valueOf()` Methods:** Overloaded static utility methods that convert any primitive type or object into a `String` representation.
 - `String.valueOf(int i)` returns a string representing the integer.
 - `String.valueOf(boolean b)` returns "true" or "false".

- `String.valueOf(Object obj)` returns `obj.toString()` or `"null"` if the object reference is null (preventing a `NullPointerException`).

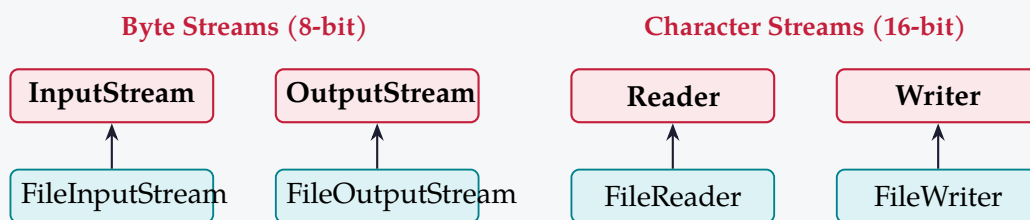
3. Java Input/Output (I/O) Streams

An I/O stream represents an abstraction of a data source or destination (e.g., keyboard, file, network socket).

3.1 Byte Stream vs. Character Stream

Metric	Byte Stream	Character Stream
Data Unit	Processes data as raw 8-bit bytes.	Processes data as 16-bit Unicode characters.
Use Case	Binary files (images, audio, zip files).	Text files (documents, configuration text).
Base Classes	<code>InputStream</code> and <code>OutputStream</code>	<code>Reader</code> and <code>Writer</code>
Key Classes	<code>FileInputStream</code> , <code>FileOutputStream</code>	<code>FileReader</code> , <code>FileWriter</code>

Java Stream Hierarchy

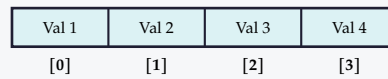
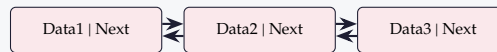


4. Java Collections Framework

The Collections Framework provides a unified architecture for storing and manipulating groups of objects.

4.1 ArrayList vs. LinkedList

Metric	<code>ArrayList</code>	<code>LinkedList</code>
Internal Structure	Dynamically resizing array.	Doubly linked list.
Access Time	$O(1)$ random access (index-based).	$O(n)$ search access (sequential traverse).
Insertion/Deletion	$O(n)$ (requires element shifts).	$O(1)$ if pointer is located.
Memory Overhead	Low (only stores elements).	Higher (stores element and prev/next links).

ArrayList vs. LinkedList Memory Layout**ArrayList (Contiguous Memory):****LinkedList (Pointer Links):**

Quick Revision — Module 4

Exception Handling, Strings, I/O Streams & Collections | OE-EC704A

Exception, File I/O & Collections Syntax

Concept	Syntax and Example Usage
Try-Catch Block	<pre>try { int a = 5/0; } catch(ArithmeticException e) { ... }</pre>
Throws Declaration	<pre>void readFile() throws IOException { ... }</pre>
Custom Exception	<pre>class MyException extends Exception { MyException(String s){super(s);} }</pre>
Buffer Reader I/O	<pre>BufferedReader br = new BufferedReader(new InputStreamReader(System.in));</pre>
Collections & Strings	Core Declarations & Queries
ArrayList Declare	<pre>ArrayList<String> list = new ArrayList<String>();</pre>
Collection Iterator	<pre>Iterator<String> itr = list.iterator(); while(itr.hasNext()) { ... }</pre>
String Concat	<pre>String s = "Biraj" + " Sarkar"; // SCP optimization</pre>
StringBuffer Append	<pre>StringBuffer sb = new StringBuffer("CGE"); sb.append("C");</pre>

One-Line Definitions

- **Checked Exception:** An exception subclass checked during compilation that requires explicit handling.
- **Unchecked Exception:** RuntimeException subclasses resulting from logical design faults.
- **Immutability:** State constraint preventing modification of heap values after initial allocation.
- **Stream:** Abstract communication pipe that handles sequential transfer of data bytes.
- **Byte Stream:** Stream subclass designed to process binary data using 8-bit units.
- **Character Stream:** Stream subclass designed to process Unicode text data using 16-bit units.
- **Collections Framework:** Framework of interfaces and classes for managing structures.
- **Iterator:** Universal collection interface cursor that iterates elements in a forward direction.

Frequently Asked / Viva Questions**Q1. What is the difference between `final`, `finally`, and `finalize`?**

`final` is a modifier indicating that a class cannot be inherited, a method cannot be overridden, or a variable cannot be reassigned. `finally` is a block accompanying a try-catch block to execute clean-up operations. `finalize` is a method invoked by the Garbage Collector before reclaiming an object.

Q2. Can we write a try block without a catch block? If so, how?

Yes, a try block can be followed by a `finally` block instead of a catch block. Additionally, Java 7 supports the **try-with-resources** statement, which automatically closes resources without requiring an explicit catch or finally.

Q3. What is try-with-resources statement?

It is a try block declared with one or more resources (e.g., streams). The resources must implement `java.lang.AutoCloseable`. They are automatically closed when exiting the try block.

Q4. Why are String objects immutable while StringBuffer objects are mutable?

Strings are cached in the String Constant Pool to save memory and ensure thread safety. `StringBuffer` is designed for dynamic character manipulation without creating multiple garbage references.

Q5. What is the difference between `equals()` and `==` when comparing strings?

`==` compares reference equality (whether they point to the exact same object reference). The `equals()` method compares value equality (whether the character sequences are identical).

Q6. What is the difference between byte stream classes and character stream classes?

Byte stream classes read and write raw 8-bit bytes (binary), whereas character stream classes read and write 16-bit Unicode characters, handling text encoding automatically.

Q7. Why do we wrap a `FileReader` inside a `BufferedReader`?

Wrapping inside a `BufferedReader` reduces disk I/O requests by reading data blocks into an internal buffer. It also provides the convenient `readLine()` method.

Q8. What is the difference between `ArrayList` and `Vector`?

`ArrayList` is non-synchronized, fast, and not thread-safe. `Vector` is synchronized, has thread-safe methods, and carries a significant performance overhead.

Q9. Why does a `LinkedList` consume more memory than an `ArrayList`?

An `ArrayList` stores elements contiguously inside a backing array. A `LinkedList` stores elements inside node wrappers that also hold memory addresses for the previous and next nodes.

Q10. How does the `throws` keyword differ from the `throw` keyword?

`throw` is used to explicitly throw a single exception instance inside a method block. `throws` is used in a method declaration to list possible checked exceptions.

Comparison: Try-Catch-Finally vs. Try-With-Resources

Metric	Standard Try-Catch-Finally	Try-With-Resources (Java 7+)
Resource Closing	Manual (must write stream close calls in finally).	Automatic (handled by compiler behind the scenes).
Boilerplate Code	High (requires nested try-catch blocks in finally).	Minimal (declared inside try parameters).
Exception Suppression	Primary exception can be lost if close fails.	Retains primary exception; close errors suppressed.
Target Classes	Any standard classes.	Only classes implementing <code>AutoCloseable</code> .

Module 5

Java Applets

1. Java Applets

An applet is a specialized Java program designed to be embedded within an HTML web page and executed on a client machine inside a web browser sandbox.

Applet Security Sandbox

Applets run in a restricted runtime environment (sandbox) to prevent malicious behaviors:

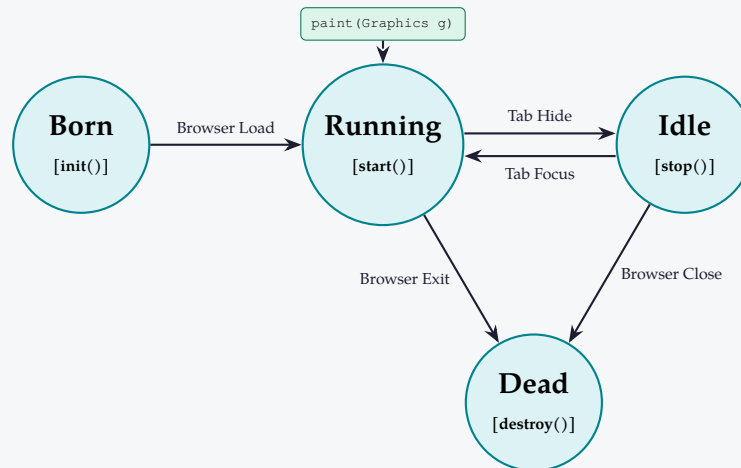
- Applets cannot read or write files on the client local disk.
- Applets cannot open network connections to any host except the one they were downloaded from.
- Applets cannot start external execution processes or execute native system code.

1.1 Applet Life Cycle

An applet defines five lifecycle callback methods managed by the browser JVM:

- **init ()**: Invoked once during loading to initialize variables and load GUI elements.
- **start ()**: Invoked after initialization or when visiting the page to resume execution.
- **paint (Graphics g)**: Invoked when the applet display needs redrawing.
- **stop ()**: Invoked when navigating away from the page to pause background threads.
- **destroy ()**: Invoked before unloading the applet to release system resources.

Applet Life Cycle State Transitions



1.2 Applet Display and Status Methods

Applets provide methods to interact with their display canvas and output messages directly to the hosting browser's status window:

- **Foreground and Background Control:** Inherited from the `Component` class.
 - `void setBackground(Color c):` Sets the background color of the applet window.
 - `void setForeground(Color c):` Sets the default color for text and graphical drawings inside the applet.
 - `Color getBackground()` and `Color getForeground():` Retrieve the current background and foreground colors.
- **Status Window Output:** The browser running the applet contains a status bar (status window).
 - `void showStatus(String message):` Displays the specified message string in the browser's status window. This is useful for user feedback (e.g., displaying coordinates, loading percentages, or event responses).

1.3 HTML Applet Tag and Parameters

Applets are executed by inserting them into an HTML document using the `<applet>` tag. The browser reads the tag, downloads the compiled bytecode, and runs it inside the local JVM.

HTML `<applet>` Tag Syntax and Attributes

```

<applet codebase="classes/" code="MyApplet.class" width="400" height="300">
  <param name="userName" value="Biraj">
  <param name="sampleRate" value="50">
  Alt text: Your browser does not support Java Applets.
</applet>

```

Core attributes of the `<applet>` tag:

- **code:** Specifies the filename of the compiled applet class (mandatory).

- **width & height:** Specify the initial display area dimensions in pixels (mandatory).
- **codebase:** Specifies the base URL or directory path containing the class files (optional).
- **alt:** Specifies fallback text to display if the browser supports the tag but cannot run it.

Passing Parameters via the <param> Tag:

- Parameters are defined in HTML using the `<param name="..." value="...">` tag inside the applet block.
- Inside the applet's `init()` method, these parameters are retrieved as Strings using: `String val = getParameter("userName");`

1.4 Applet vs. Standard Application

Feature	Applet Class	Standard Java Application
Execution Target	Embedded in HTML; runs inside browser JVM.	Standalone execution on the client OS.
Main Entry Method	No <code>main()</code> entry. Uses lifecycle methods.	Requires a <code>public static void main</code> entry.
Security Rules	Heavily sandboxed (restricted access).	Full access to local hardware and file systems.
Superclass Class	Must extend <code>java.applet.Applet</code> .	Does not require a specific superclass.

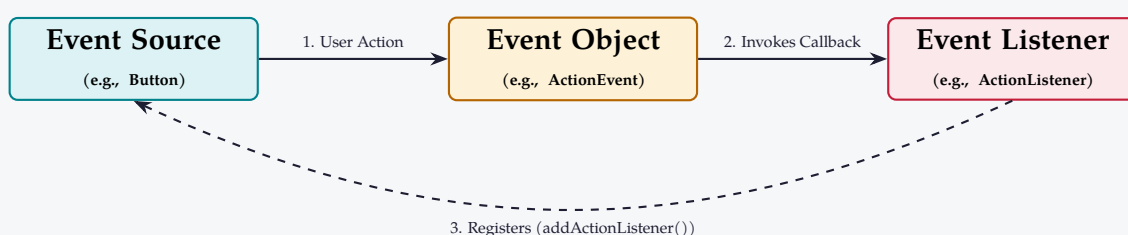
2. Delegation Event Model

Java event handling is built on the **Delegation Event Model**, which separates the user interface controls from the logic handling user events.

Model Components

- **Event Source:** The GUI component that generates the event when acted upon (e.g., `Button`).
- **Event Object:** Encapsulates details of the event state change (e.g., `ActionEvent`).
- **Event Listener:** An interface target that receives and processes event notifications.

Delegation Event Model Architecture



2.1 Adapter Classes

An event listener interface may define multiple abstract methods (e.g., `WindowListener` defines seven). Implementing it directly forces you to provide empty overrides for all of them.

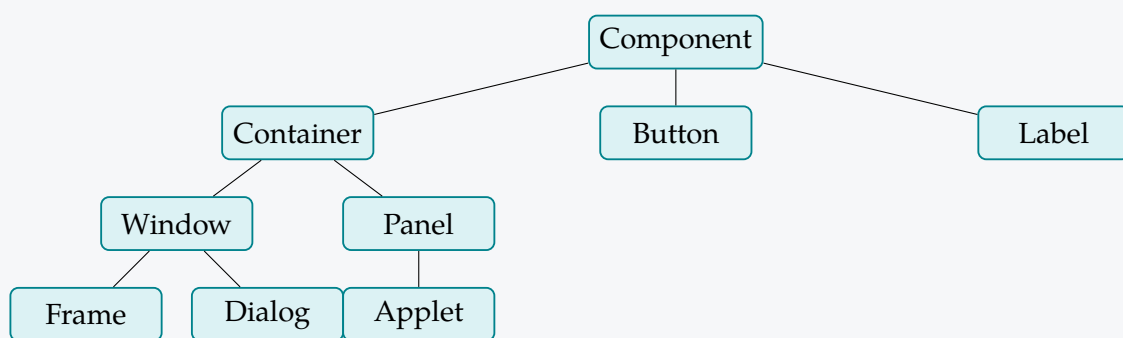
Adapter Class Solution

An **Adapter Class** is a helper class that implements a listener interface with empty default method overrides. By extending the adapter class (e.g., extending `WindowAdapter`), you only override the specific callback methods you need.

3. AWT (Abstract Window Toolkit)

AWT is Java's original platform-dependent GUI library. It uses native system peers to render controls, resulting in a look-and-feel that matches the host OS.

AWT Component Class Hierarchy



3.1 AWT vs. Swing

Metric	AWT (Heavyweight)	Swing (Lightweight)
Rendering Engine	Uses host OS peer components.	Rendered purely in Java (custom painting).
Platform Style	Platform-dependent look-and-feel.	Platform-independent look-and-feel.
Speed	Fast (delegates rendering to host).	Slower (rendered by JVM engine).
Class Package	Lives in <code>java.awt.*</code>	Lives in <code>javax.swing.*</code>

3.2 Layout Managers

Layout managers control the sizing and positioning of child components inside a container:

- **FlowLayout (Default for Panel/Applet):** Places components in a row, wrapping when the boundary is reached.
- **BorderLayout (Default for Frame/Window):** Places components in five regions: North, South, East, West, Center.

- **GridLayout:** Places components in a uniform grid of equal-sized rows and columns.

Quick Revision — Module 5

Applets, Event Handling & AWT Controls | OE-EC704A

HTML Applet & Event Handling Syntax

Concept	Syntax and Example Usage
HTML Applet Tag	<code><applet code="MyApplet.class" width="300" height="200"></applet></code>
Applet Parameters	<code><param name="message" value="Hello World"></code>
Parameter Query	<code>String val = getParameter("message");</code>
Applet paint Override	<code>public void paint(Graphics g) { g.drawString("Test", 20, 20); }</code>
AWT & Events	Registration & Design Declarations
AWT Button Registration	<code>Button btn = new Button("Submit"); btn.addActionListener(listener);</code>
Anonymous Inner Class	<code>btn.addActionListener(new ActionListener() { ... });</code>
Set Frame Layout	<code>setLayout(new BorderLayout()); add(btn, BorderLayout.NORTH);</code>
AWT Frame Show	<code>Frame f = new Frame("Title"); f.setSize(300, 300); f.setVisible(true);</code>

One-Line Definitions

- **Applet:** A small Java program executed inside a web browser container.
- **Delegation Event Model:** An event framework separating event detection from handler logic.
- **Event Source:** A UI component that registers listeners and fires events on state changes.
- **Event Listener:** An interface declaring callback methods to handle specific events.
- **Adapter Class:** A helper class with default empty implementations of a listener interface.
- **Anonymous Inner Class:** An unnamed inline class used to implement listener interfaces.
- **AWT:** A platform-dependent GUI library utilizing host OS peers for rendering.
- **Layout Manager:** A layout interface deciding the spatial distribution of child components.

Frequently Asked / Viva Questions**Q1. Which class does every Java Applet extend? Which package contains it?**

Every Applet extends `java.applet.Applet` (or `javax.swing.JApplet` for Swing). The class resides inside the `java.applet` package.

Q2. Why does an applet not contain a `main()` method?

Applets do not run as standalone applications. They are executed by a web browser container, which handles loading and calls their lifecycle methods.

Q3. What is the difference between `init()` and `start()` methods?

`init()` is called once when the applet is first loaded to handle initialization. `start()` is called every time the user visits or returns to the page.

Q4. What is the purpose of the `paint(Graphics g)` method? Who calls it?

The `paint()` method renders the applet graphics (text, lines, shapes) on the screen. It is called by the browser JVM when the applet first displays or needs redrawing.

Q5. How do you pass parameters from HTML to an Applet?

Using the `<param>` tag inside the `<applet>` tag in HTML, and querying it in the applet using the `getParameter(String name)` method.

Q6. What is the difference between `getCodeBase()` and `getDocumentBase()`?

`getCodeBase()` returns the URL of the directory containing the applet's compiled class files. `getDocumentBase()` returns the URL of the HTML document embedding the applet.

Q7. Why are AWT components referred to as heavyweight controls?

AWT controls are coupled to the underlying operating system. Each AWT component is associated with a native peer window, consuming significant memory and OS resources.

Q8. What is the purpose of an Adapter class in event handling?

It simplifies listener implementation. By extending an adapter class, you only override the specific event handler methods you need, rather than providing empty overrides for all methods.

Q9. How does `FlowLayout` arrange components? What is its default alignment?

`FlowLayout` arranges components in a single line from left to right. When it runs out of space, it wraps them to the next line. Its default alignment is centered.

Q10. Why did AWT become deprecated in favor of Swing?

AWT's platform-dependent rendering led to inconsistent layouts across operating systems. Swing introduced lightweight, platform-independent controls rendered entirely in Java.

Applet Lifecycle Methods Transition Matrix

Method	Invocation Frequency	Primary Purpose
init ()	Invoked once when first loaded.	Initializes variables, loads fonts, images, and GUI elements.
start ()	Invoked on load and page returns.	Resumes applet execution, starts animations or threads.
paint ()	Invoked on display and redraw requests.	Renders text, shapes, and images to the screen coordinate system.
stop ()	Invoked on page exit or tab switch.	Suspends threads, pauses animations to save CPU resources.
destroy ()	Invoked once before unloading.	Reclaims resources, kills threads, and cleans up memory.